Veri Yapıları Ve Algoritmalar

online.yildiz.edu.tr Ders duyuru içerisinde ders içeriği mevcut

10:05 gece ders başlıyor: 45 dk ders 15 dk ara

bu derste not tutmak önemli ▽

2 vize	4 ödev	1 proje	final
%20	%12	%8	%40

Devam Zorunluluğu Yok ▽

Lablarda bazen konu bazen uygulama bazende ödev verilip ödev kontrolü yapılacak. 1-2 hafta dışında aktif kullanılacak.

Sınavlar algoritmik şekilde olucak Geçmiş sınavlara dikkat et.

Derse gelmeden önce syllibus'a bak.

1. Hafta

Algorithm Design Criteria

* Correctness
* Easy to implement
* Easy to understand
* Easy to modify
* Time efficiency
* Space efficiency

Algorithm Presentation

* flowchart
* Pseudo Code
* Programming Language
* Dial presentation

Programming

* Ansi C standard
* Global Variables (Do not use)
* Variable Names, function Names
* Modular Programming
* Dynamic Memmory Allocation
* Comment Lines

Program = Algorithm + Data Structers

Data Structers

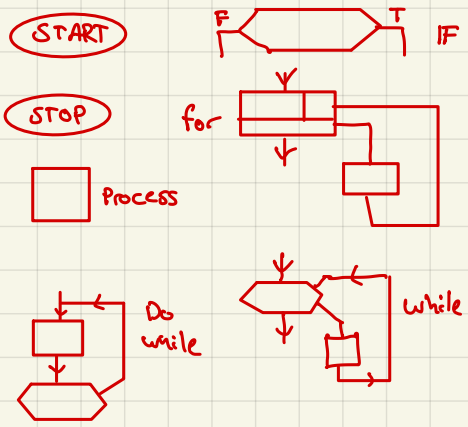
* Arrays
* Linked Lists
* Stack
* Queue
* Tree
* Graphs
* Heap

decrease and conquer

Search Algorithms

* Linear $O(n)$
* Binary search $\log_2 n$
* Hashing or Hashing?
* Jump search $O(\sqrt{N})$
* Binary search tree
* Interpolation search
* Exponential search

Flowchart Symbols



Basic Operations

* Sort
* Search
* Insert
* Delete
* Update

Sorting Algorithms

* Bubble sort
* Selection Sort
* Quick Sort
* Merge Sort
* Heap Sort
* Insertion sort
* Shell sort
* Radix sort

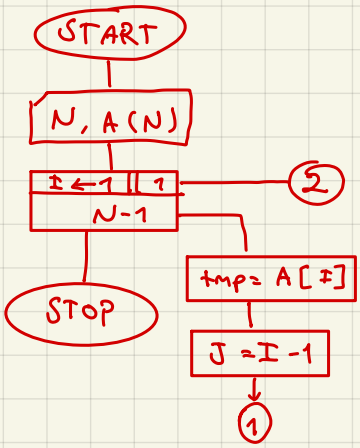
} Comperison Sort Algorithms

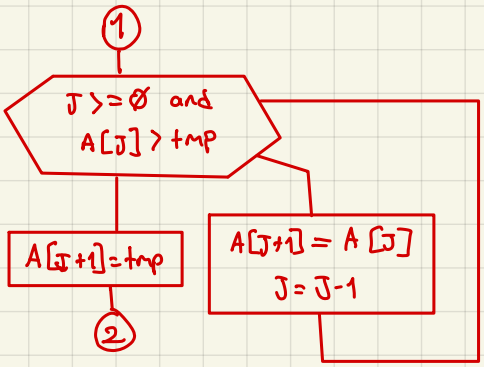
* Counting sort (LSD, MSD) → Counting sort Algorithm

Insertion Sort

5	1	4	3	2	7	6

→ 1 5 4 3 2 7 6
→ 1 4 5 3 2 7 6
→ 1 3 4 5 2 7 6
→ 1 2 3 4 5 7 6
→ 1 2 3 4 5 6 7

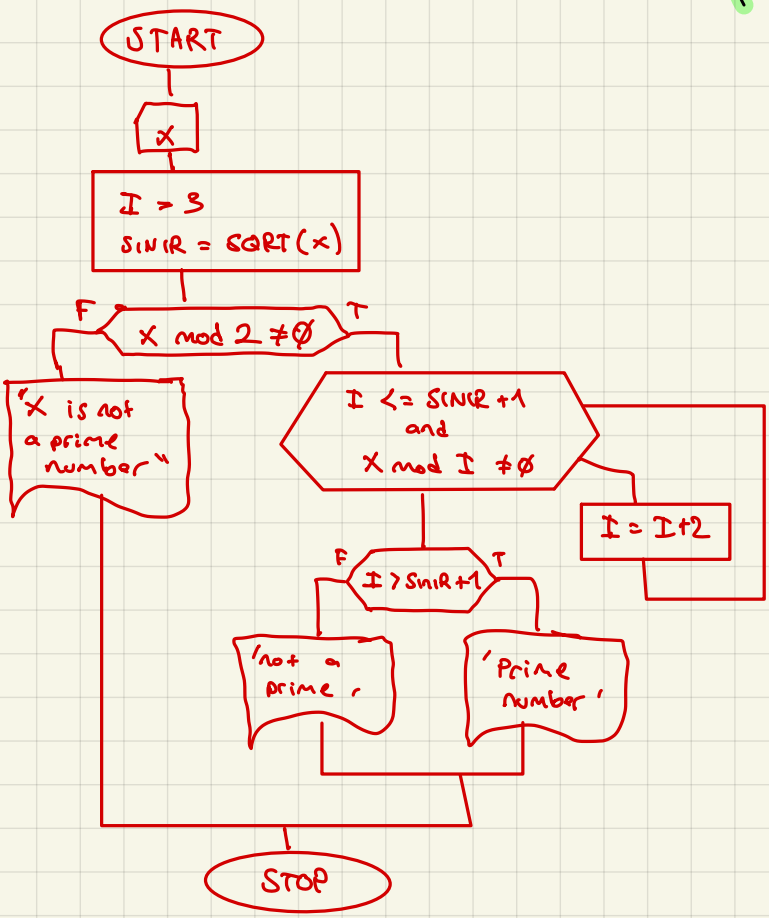




Worst-case time complexity : $O(n^2)$
Best-case time complexity : $O(n)$

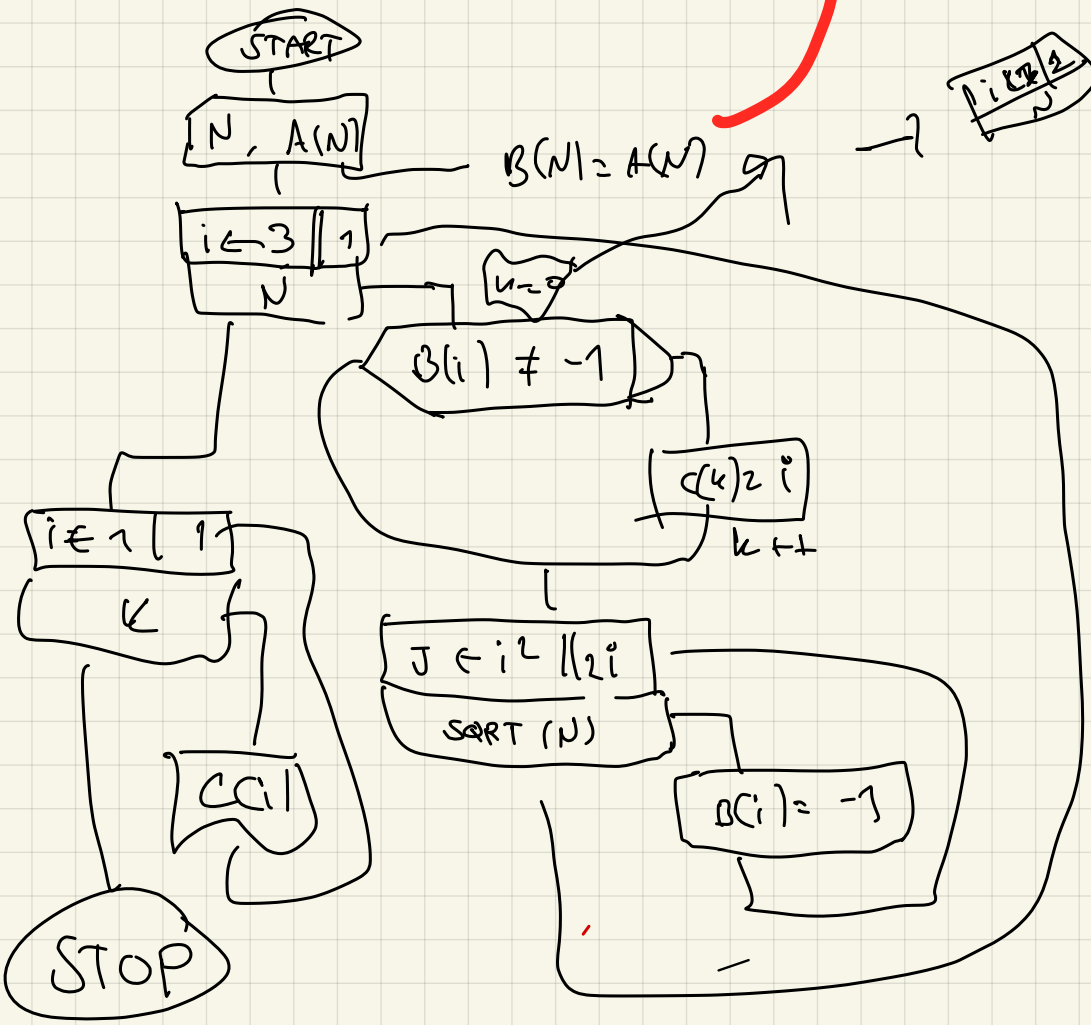
Finding Prime Numbers

* **Naive Solution** (Bread force Approach) (asal sayı bulurken sayının kareköküne kadar olan sayıları kontrol etmek asal olup olmadığını anlamakta yeterlidir.)



Sieve of Eratosthenes Algorithm

haftaya araştır. Akışını Çiz gel ▽



Algorithm Analysis

- Time complexity
- Space Complexity

Basic Operations

- Comperission Op.
- Assignment Op.
- Arithmatic Op.
- Intmization Op.

Search algoritmalarına bak

Example: Find maximum value on array.

Pseudo Code:

Basic OP	
2	current Max ← A [0]
2+n	for i ← 1 to n-1 do
2(n-1)	if A[i] > current Max then
2(n-1)	current Max ← A[i]
2(n-1)	increment counter
1	return current Max

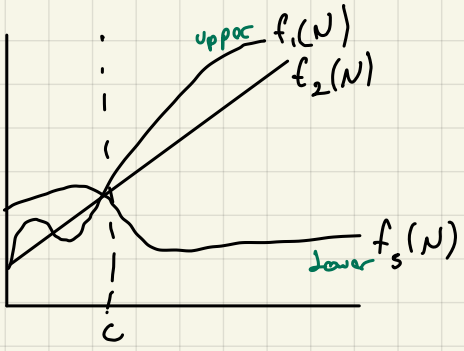
$O(7n-1) \rightarrow O(n)$

Asymptotic Notation

Big Oh (O) Notation upper bound

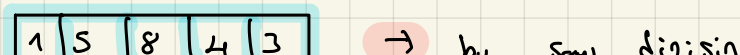
Big Theta (Θ) Notation Thight bound

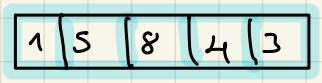
Big Omega (Ω) Notation lower bound



Functions Growth Rate

$1 \quad \log(N) \quad N \quad N\log(N) \quad N^2 \quad N^3 \quad 2^N \quad N!$





→ bu sayı dizisinden toplam olarak "13" ü bulmak için hangi sayıları seçmemiz gerektiğini bulan algoritmanın karmaşıklığı 2^N 'dir. Alt kümeleri kontrol etmemiz gerektiği için!

Recursive Bak

Recursive vs Iteration

Criteria	Iteration	Recursion
- Made Of Implementation	Using Loops	function call itself
- State	Defined by the control variables value	Defined by the parameter values stored in stack
- Progression	The value of control variable moves toward the value in condition	The function state converges towards the base case
- Termination	Loop ends when control variable's value satisfies the condition	Recursion ends when base case becomes true
- Code Size	Iterative code tends to be bigger in size	Recursion decreases the size of code
- No Termination State	Infinite Loops use CPU cycles	May cause stack overflow error or it might crash the system.
- Execution	faster	Slower

Recursive Pros and Cons

- İterasyona göre daha yavaş çalışır
+ Daha okunaklı ve az kodla yazılması
+ Bazı problemlerin algoritmik tasarımı daha kolaydır

Factorial Program Using Recursion

```
int factorial (int n) {
    if (n<=1)
        return 1;
    return n * factorial (n-1);
}
```

X^Y

10^3 → 10 10 10

```
int fonksiyon (int X, int Y) {
    if (Y==0)
        return 1;
    return X * fonksiyon (X, Y-1);
}
```

Verilen Sayı kaç Basamaklı

```
int basamak (int x) {
    if (x / 10 == 0)
        return 1;
    return 1+ basamak( X / 10 );
}
```

Optimizasyon Gözüm

```
int basamak (int x) {
    static int count = 1;
    if (x>9){
        count ++;
        basamak (x/10)
    }
    return count;
}
```

Fibonacci Numbers → 0 1 1 2 3 5 8 13 21

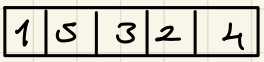
```
int fibonacci (int n) {
    int a, b;
    if (n<=1)
        return n;
    a = 0 , b =1;
    for (i=2 ; i<=n; i++){
        c=a+b
        a=b;
        b=c;
    }
}
```

Iterative

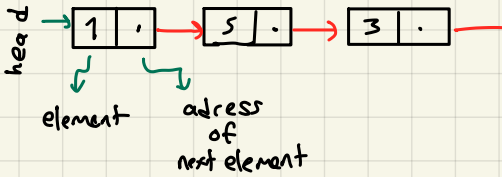
Recursive

```
int fibonacci (int n) {
    if (n<=1)
        return n;
    return fibonacci (n-1) + fibonacci (n-2);
}
```

Linked Lists



→ array → Direct access (Adressing)
→ Contigius Memory Usage (stack) gibi

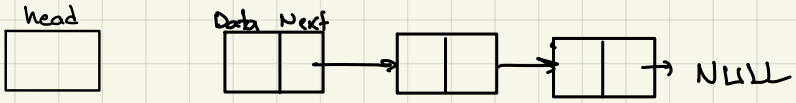


→ Linked List → Non-Contigius Memory Usage (heap) gibi

★ ekleme ve silme işleminin yoğun olduğu ve bellekte parça parça boşlukların olduğu durumlardan linked list'i kullanmak akıllıcadır.

★ Çift array ya da nx2 lik matrisler ile linkli liste kullanımı yapılabilir.

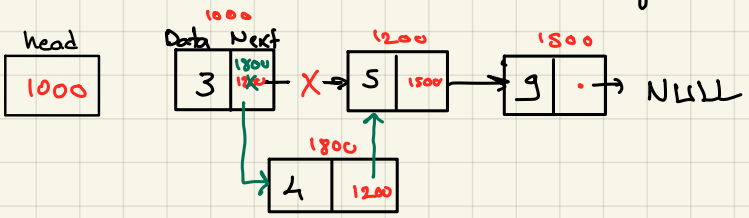
Type of Linked Lists

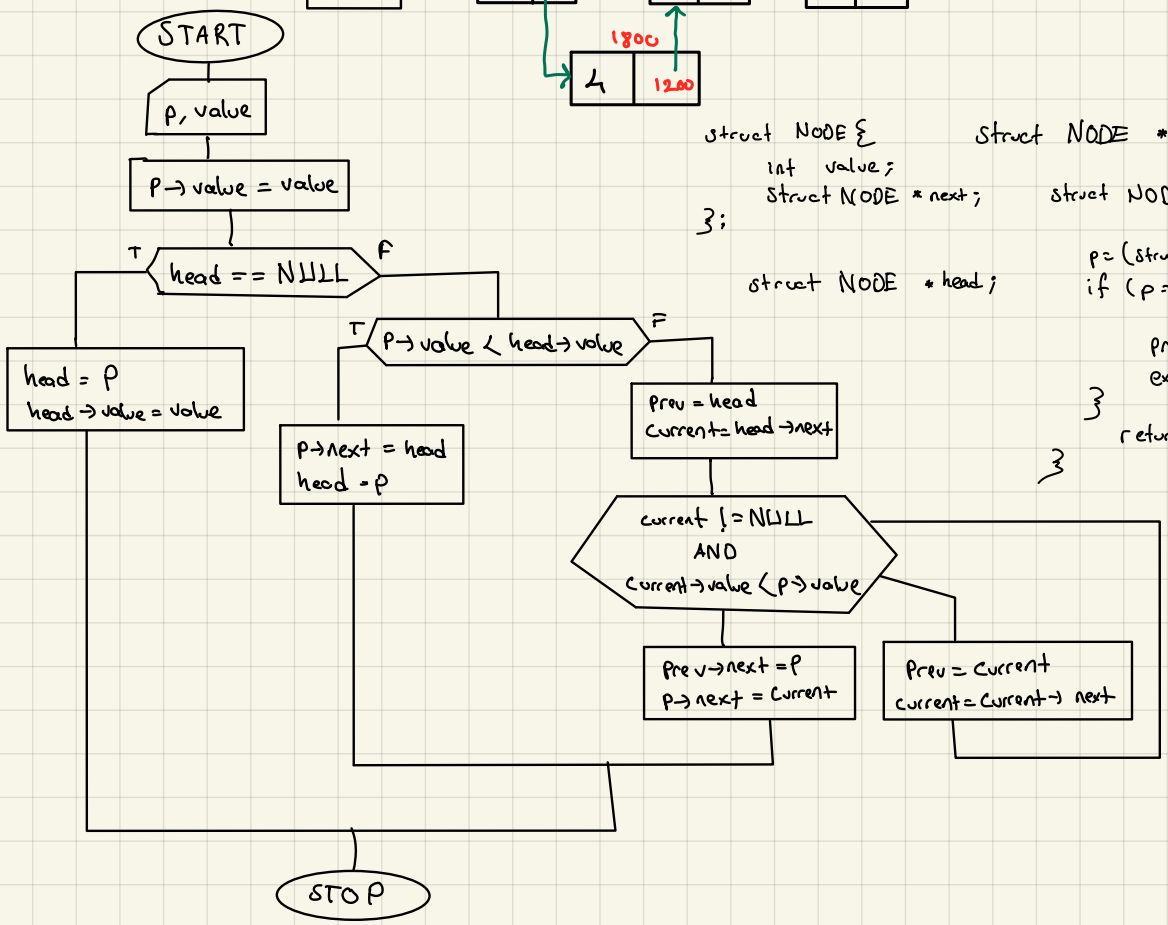


1. Single Linked Lists
2. Doubly Linked List
3. Circular Linked List
4. Circular Doubly Linked List

Insert Operation

★ Sırasız bir linkli listede yeni eleman en başa koyulur.



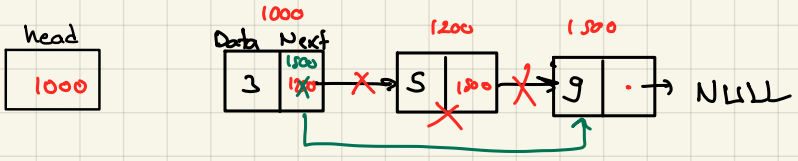


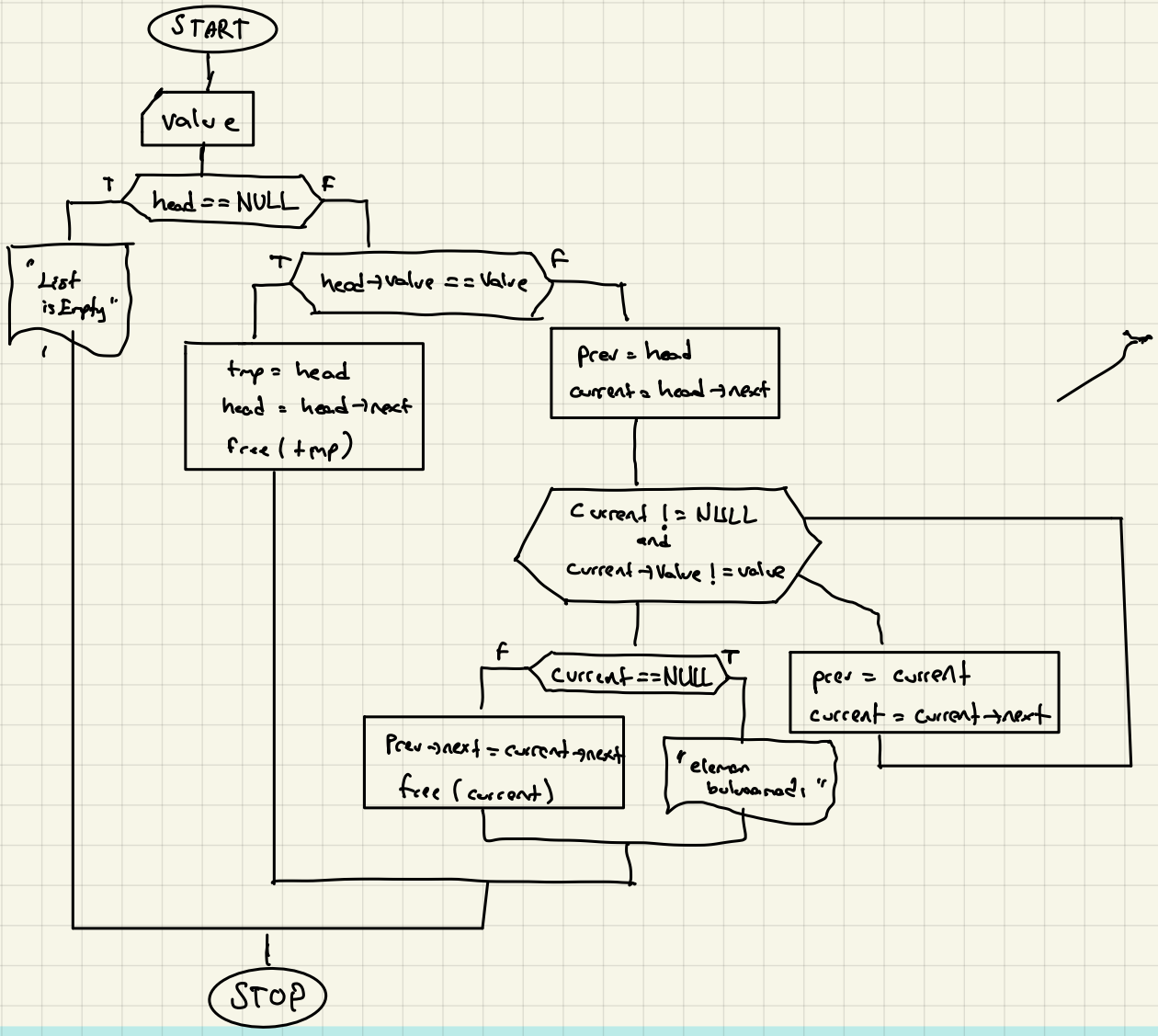

```
struct NODE {
    int value;
    struct NODE *next;
};

struct NODE *head;
```

```
struct NODE *creatNode() {
    struct NODE *p;
    p = (struct NODE *) malloc (sizeof (NODE));
    if (p == NULL) {
        printf ("Allocation error");
        exit (0);
    }
    return p;
}
```

Delete Operations





Linked List Operations

- Traversal
- Insertion
- Deletion
- Search
- Update
- Sort

Linked list vs Array

Array	Linked List
- Random Access	- Sequential Access
- Contigius Memory usage	- Non-Contigious Memory usage
- Slower on insertion deletion	- faster on insertion deletion
- Static Memory Allocation	- Dynamic Memory All..
- Single, Two, multi dimension	- Single, Doubly, Circular
- Stack	- Heap
- Cache Locality	

⇓

bellekte belli kısım cache'e alındığı için
linkli listede dağınık olduğu için array avantajlı

APPLICATIONS on LINKED LISTS

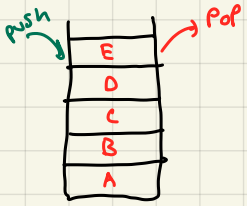
1-) Stack, Queue
2-) Graph
3-) Hashing
4-) Dynamic Memory Allocation
5-) Maintaining Directory of Names
6-) Representing Sparse Matrices

Real-Life

* Image viewer
* Previous and next page in web browser
* Music player
* Operating System (Process Management)

STACK (yığın) (Abstract Data Type)

- first in Last out FILO
- Last in first out LIFO

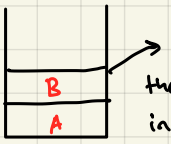


Applications

1. Evulation of arithmathic Op.
   A+(B-C(D*E/(G)-F)
2. Recursion function usage
3. Reversing String
4. Backtracking (Depth First Search)
5. Paranthesis Matching
6. Conversion of Decimal to other Number Systems

Stack Operations

- push()
- pop()
- isFull()
- isEmpty()
- peek()
- top()

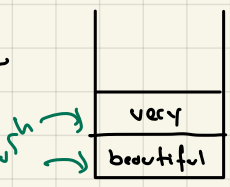


get the value of (top) the last element in the stack

Stack Usage

The weather is very beautiful
2 1

The weather is very
The weather is



Stack Usage In Compilers

1. -(a+b(error
2.)a+b(error
3. a*(b+c() error
4. ((a+b)*c) ✓

Syntax Error

Stack Operations

```
#define MAX 10
typedef struct{
    int item[MAX];
    int top;
}STACK;
void initStack(STACK *s){
    s->top = 0;
}
```

```
int isEmpty(STACK *s){
    if(s->top == 0)
        return 1;
    else
        return 0;
}
int isFull(STACK *s){
    if(s->top == MAX)
        return 1;
    else
        return 0;
}
```

```
int push(int x, STACK *s){
    if(isfull(s))
        return 0;
    else{
        s->item[s->top++] = x;
        return 1;}
}
int pop(STACK *S, int *x){
    if(isEmpty(s))
        return 0;
    else{
        --s->top;
        *x = s->item[s->top];
        return 1;}
}
```



Stackte bir sonraki nokta gösterildiği için aradaki pop edilir

```
int top (STACK *s, int *x){
    int adr;
    if (isEmpty(s))
        return 0;
    else {
        adr = s->top -1;
        *x = s->item[adr];
        return 1;}
}
```

Arithmatic Expression Evolution

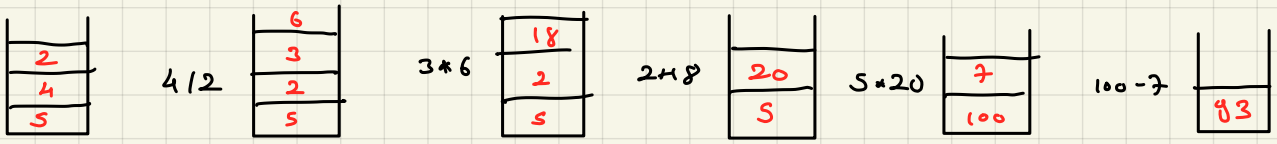
- Infix → A+B
- Prefix → +AB
- Postfix → AB+

Precedence

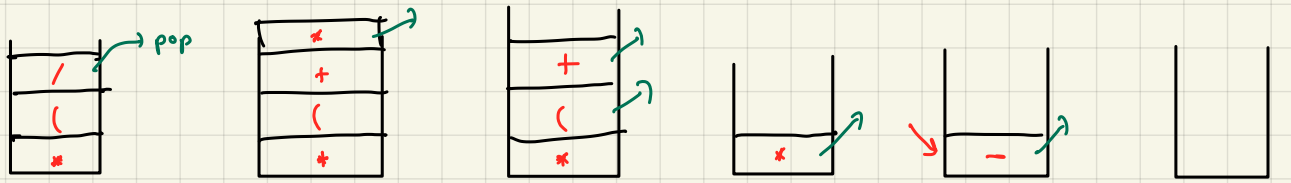
(
* /
+ -

5 * (4 / 2 + 3 * 6) - 7

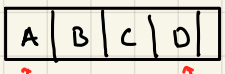
operands



operators

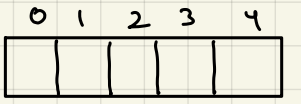


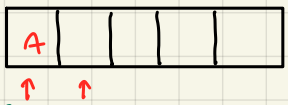
Queue (Kuyruk) (Abstract Data Type) iki uclu STACK

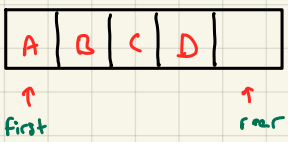


Queue — First in First out.

- Double Ended Queue
- Priority Queue
- Circular Queue







→ Implementasyona göre boş alanı gösterebilir.

Queue Operations

- isEmpty()
- isFull()
- enQueue()
- deQueue()
- initQueue()
- peek()

Queue Implementation on Linked Lists

- ?

```
int isEmpty() {
    if( front == -1)
        return 1;
    else
        return 0;
}
```

```
int isfull() {
    if ( rear == Max-1)
        return 1;
    else
        return 0;
}
```

```
int deQueue() {
    if (isEmpty())
        return 0;
    else {
        *X = q[Front++];
    }
    return 1;
}
```

```
int enQueue(int value) {
    if (isfull())
        return 0;
    else {
        rear ++;
        q[rear] = value;
    }
    return 1;
}
```

```
void initQueue() {
    int front = -1, rear -1;
}
```

```
int isfull() {
    if ( (Front == rear +1) || (front == 0 && rear == SIZE-1))

        return 1;
    else
        return 0;
}
```

```
int enQueue(int value) {
    if (isfull()) {
        return 0;
    }
    else if (front == -1)
        front = 0;
    rear = (rear +1) % SIZE;
    q[rear] = value;

    return 1;
}
```

```
int deQueue(int *X) {
    if ( isEmpty())
        return 0;
    else {
        *X = q[front];
        if ( front == rear ) {
            front = -1;
            rear = -1;
        }
        else
            front = (front +1) % SIZE;
        return 1;
    }
}
```

Double Ended Queue

* Input Restricted dequeue (D.E.Q.)
  input restricted at a single end but allows deletion at both the ends
* Output Restricted Deque
  output is restricted at a single end but allows insertion at both the ends

Comparission STACK vs Queue

	STACK	Queue
* Working Principle	LIFO	FIFO
* Structure	Same end used to insert and delete	One end is used for insert Another end used for deletion
* Number of Pointers	one	two
* Operations	Push, Pop	Enqueue, Dequeue
* Variants	It does not have	Circular, Priority, Double Ended Queue
* Implementation	Simpler	Comperatively complex

Linear Data Structure

* Array
* Linked List
* STACK
* Queue

Non-Linear Data Structure

(Hierarchical Data Structure)

* Graphs
* Trees

	Linear Data Structure	Non-Linear Data Structure
Basic	The data item are arrayed in an orderly manner where the elements are attached adjacently	It arranges the data in a sorted order and there exists a relationship between the data elements
Ease of Implementation	Simpler	Complex
Levels involved	Single	Multiple

Terminology

* Path
* Root
* Parent
* Child
* Leaf
* Subtree
* Visiting
* Traversing
* Level
* forest
* Depth
* Height

Tree Data Structure



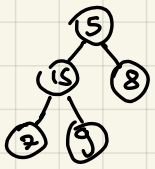
Tree App. in Real-life

- File management system
- Compression Algorithms (Huffman Alg.)
- Databases
- Compilers
- Priority Queues
- AI (Decision Tree, Random forest)
- Indexing multi-dimensional information such as geographical coordinations, rectangles or polygons.

Types of Tree

- Binary tree
- Binary Search tree
- AVL tree
- B-tree
- R-tree
- Interval tree
- D-ary heap
- Heap tree

Tree Traversal



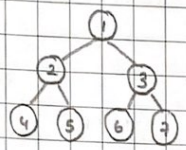
- Inorder Traversal — left subtree → node → right subtree — 7 → 15 → 9 → 5 → 8
- Preorder Traversal — node → left subtree → right subtree — 5 → 15 → 7 → 9 → 8
- Postorder Traversal — left subtree → right subtree → node — 7 → 9 → 15 → 8 → 5

```
struct node {
    int data;
    struct node * left;
    struct node * right;
}
```

```
void inOrderTraversal(struct node * root){
    If (root == NULL)
        return;
    inOrderTraversal(root → left);
    printf("%d", root → data);
    inOrderTraversal(root → right);
}
```

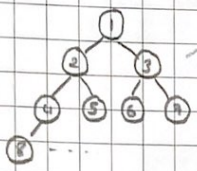
```
void preOrderTraversal(struct node * root){
    If (root == NULL)
        return;
    printf("%d", root → data);
    preOrderTraversal(root → left);
    preOrderTraversal(root → right);
}
```

```
void postOrderTraversal(struct node * root){
    If (root == NULL)
        return;
    postOrderTraversal(root → left);
    postOrderTraversal(root → right);
    printf("%d", root → data);
}
```



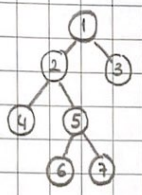
Perfect Binary Tree

It is a type of binary tree which every internal node has exactly two child nodes and all the leaf nodes are at the same level



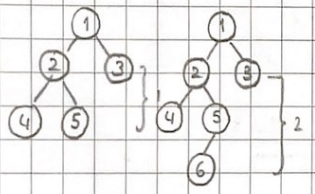
Complete Binary Tree

It is a binary tree in which all the levels are completely filled except possibly the lowest one, which is filled from the left



Full Binary Tree

It is a binary tree in which every parent / internal node has two or no children

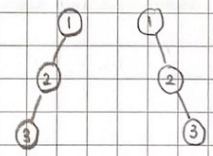


Balanced Binary Tree

Uç noktalar arası birden fazla olmayacak



Degenerated or Pathological Tree



Skewed Binary Tree

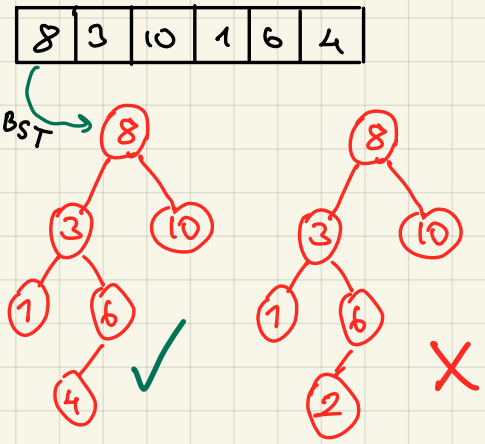
Dilman

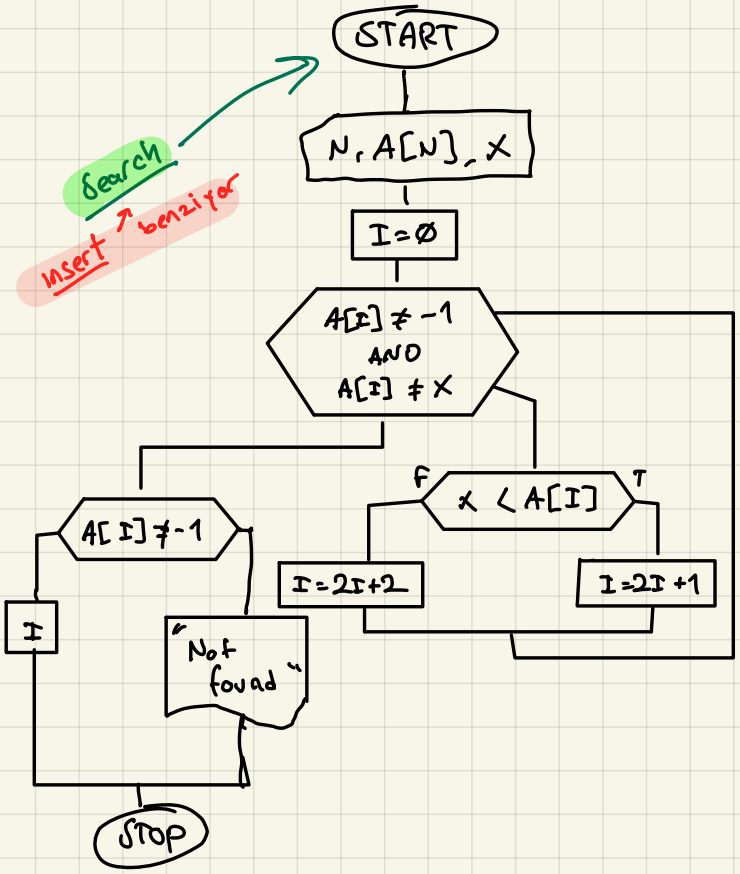
Binary Search Tree

- Search
- Insert
- Delete

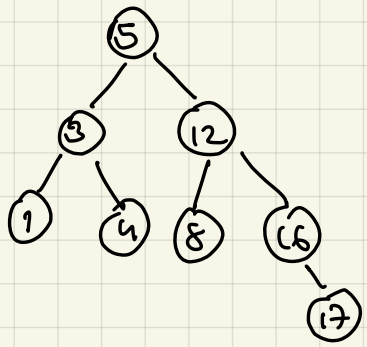
Ø → 2k+1, 2k+2

1 → 2k, 2k+1





Delete



Case I = Delete Leafnode

Case II = Delete internal node with a single child.

Case III = Delete internal node (right, left child.

Case IV = Not found

Case III:

1-) Get the inorder succesor of that node
2-) Replace the node with the inorder succesor
3-) Remove the inorder succesor from its original position.

inorder succesor → kendisinden sonra gelen en büyük eleman

Inorder succesor of a node is the next node in inorder traversal of a Binary Tree

Inorder succesor of an input node can be also defined as the node with the smallest key greater than the key of the input node

1-) If the right subtree of a node is not NULL, than succesor lies in the right subtree.
2-) If " " " " " is Null, start from root;
travel down the tree, if a node's data is greater than root's data than go right side, otherwise go left side.

```
Struct node * deleteNode ( struct node * root, int key){

    if ( root == Null ) return root ;

    if ( key < root -> key )
        root -> left = deleteNode ( root -> left , key);
    else if ( key > root -> key)
        root -> right = deleteNode (root -> right, key);
    else {
        if ( root -> left == NULL) {
            struct node * temp = root -> right;
            free ( root) ;
            return temp ;
        }
        else if ( root -> right == NULL) {
            struct node * temp = root -> left ;
            free ( root) ;
            return temp ;
        }
        Struct node * temp = minValue Node ( root -> right) ;
        root -> key = temp -> key;
        root -> right = deleteNode ( root -> right, temp -> key);
    }
    return root;
}

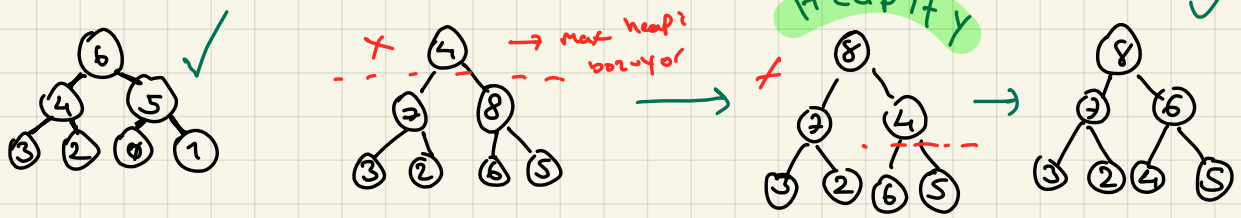
Struct node * minValue Node ( struct node * node){

    Struct node * current = node;
    while ( current && current -> left != NULL)
        current = current -> left ;
    return current;
}
```

Heap Tree

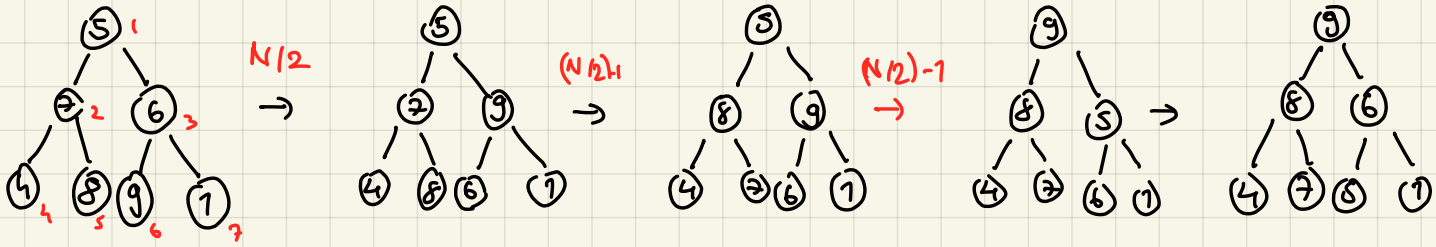
- Complete Binary Tree
- Max-Heap → A[parent(i)] ≥ A[i]
- Min-Heap → A[parent(i)] ≤ A[i]

Array : | 6 | 4 | 5 | 3 | 2 | 0 | 1 |



- Insert → sona ekle ve yine heapify yap
- Search
- Delete → Silmek istediğin node'u sona at, size'ı 1 düşür heapify yap

Max-Heap Tree Conversion




```
Void buildMaxHeap(int dizi[]) {
    int i;
    for (i=N/2; i>1; i--)
        maxHeapify(dizi, i, N);
}
```

Heap Sort kendin bak

```
void maxHeapify(int dizi[], int i, int N) {

    int left = 2*i;
    int right = 2*i+1;
    if((left <= N) && dizi[left] > dizi[i])
        largest = left;
    else
        largest = i;
    if((right <= N) && dizi[right] > dizi[largest])
        largest = right;
    if(largest != i) {

        swap(dizi[i], dizi[largest]);
        maxHeapify(dizi, largest, N);
    }
}
```

Haftaya Priority Queue

Delete Heapify bak

Heap Sort:

```
void heapSort( int arr[], int n ){
    for ( i=n/2-1 ; i>=0 ; i--)
        heapify(arr, n, i);
    For ( i=n-1 ; i>=0 ; i--){
        swap (&arr[0], &arr[i]);
        heapify ( arr, i, 0);
    }
}
```

```
void swap ( int *a, int *b){
    int temp;
    temp = *a;
    *a = *b;
    *b = temp;
}
```

Heap Sort Complexity

Best	O(nlogn)
Average	O(nlogn)
Worst	O(nlogn)
Space	O(1)
Stability	NO

eğer 2 aynı değer varsa ve bunlar sıralan dık tan sonra yerleri değişiyorsa Stable değildir.

#16 ahmet
#17 ahmet
Non Stable

Priority Queue

- Linked List
- Array
- Heap Tree
- Binary Search Tree

- Process Management and Interrupt Handling
- Djikstra Algorithm
- Data Comperission (Huffman Coding)
- Queue on Banking Operations
- Airplane Queues

Priority Queue Operations

- Insert
- Delete
- Peek

Operation Complexity

	Peek	Insert	Delete
• Linked list	O(1)	O(n)	O(1)
• Binary Tree	O(1)	O(logN)	O(logN)
• BST	O(1)	O(logN)	O(logN)

Priority Queue Using Heap Tree

Binary tree yerine heap tree tercih edilir
Complexity açısından aynıdır

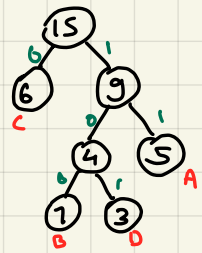
Huffmann Coding

B C A A D D D C C A C A C A C

1 6 5 3
B C A D

⟶

1 3 5 6
1 3 4 5 6
1 3 4 5 6 9
1 3 4 5 6 9 15



C	0	x 6	6
A	11	x 5	10
B	100	x 1	3
D	101	x 3	+ 9
			28

Algorithm

1. Calculate the frequency of each char in the string
2. Sort the chars in increasing order of the frequency.
3. Make each unique char as leaf node.
4. Create an empty node n. Assign the min frequency to the left child of n and assign the second minimum frequency to the right child of n. Set the value of the n as the same of the above two min frequencies.

5. Remove these two min frequencies from Q and add the sum into the list of frequencies.
6. Insert node n into the tree.
7. Repeat steps 3 to 5 for all characters.

Example:

A → 5	1100	→	20
B → 9	1101	→	36
C → 12	100	→	36
D → 13	101	→	39
E → 16	111	→	48
F → 45	0	→	+ 45
			224

5 9 12 13 16 45
5 9 12 13 14 16 45
5 9 12 13 14 16 25 45
5 9 12 13 14 16 25 30 45
5 9 12 13 14 16 25 30 45 55 → 100

